

KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

Department of Computer Science & Engineering | Complete Practical Execution Manual

Software Engineering Lab Internal I (23CC501PC / 23CC505PC) • Sets 1 through 9
Includes detailed editor instructions for Eclipse IDE, Git Bash, Docker Desktop, and Web Browser.

SET 1: Hospital Management System (HMS)

Subject Code: 23CC505PC | Faculty: K. Yasaswini | Branch: CSE - A | Repository: <https://github.com/ssvkotamraju/HMS.git>

Part I: Maven Build Execution & Troubleshooting (40 Marks)

1. Clone the HMS Source Code and Import into Eclipse

STEP 1: IN GIT BASH

```
git clone https://github.com/ssvkotamraju/HMS.git
cd HMS
```

STEP 2: IN ECLIPSE IDE

- Click **File** → **Import...** from the top menu bar.
- Expand the **Maven** folder → select **Existing Maven Projects** → click **Next >**.
- Click **Browse...** and select your cloned root folder where `pom.xml` is located.
- Ensure the checkbox next to `/pom.xml` is ticked → click **Finish**.
- Verify in Eclipse: In **Package Explorer**, confirm the project folder displays an **[M]** icon overlay and the **Maven Dependencies** library is present.

2. "'mvn' is not recognized" Error Diagnosis and Environment Setup

WINDOWS ENVIRONMENT VARIABLES SETUP

- Press **Win + S**, type `env`, and click **Edit the system environment variables**.
- Click **Environment Variables...** at the bottom right.
- Under **System Variables**, click **New...**:
 - Variable name: `JAVA_HOME` | Variable value: `C:\Program Files\Java\jdk-17`
 - Variable name: `MAVEN_HOME` | Variable value: `C:\apache-maven-3.9.6`
- Select **Path** under System Variables → click **Edit...** → **New** → add: `%MAVEN_HOME%\bin`.
- Click **OK** on all dialogs.

IN GIT BASH (VERIFICATION)

```
mvn -version
```

3. Unresolved Dependencies in Eclipse & pom.xml Fixes

IN ECLIPSE IDE

- Open the **Problems View** (**Window** → **Show View** → **Problems**) to inspect missing JAR coordinates.
- In **Package Explorer**, double-click `pom.xml`. At the bottom of the editor window, click the `pom.xml` tab (to view XML source).
- Correct any typos in `<groupId>`, `<artifactId>`, or `<version>` tags.
- Right-click the project folder in Package Explorer → select **Maven** → **Update Project...** (**Alt + F5**).
- Tick **Force Update of Snapshots/Releases** → click **OK**.

4. Maven Compilation Failure Troubleshooting

IN ECLIPSE IDE

- Right-click project in Package Explorer → select **Run As** → **Maven build...**
- In the **Goals** field, enter: `clean compile -e -X` → click **Run**.
- Inspect the **Console View**: `-e` prints detailed exception stack traces pointing to the exact Java file and line number failing compilation; `-X` enables debug logging for classpath and compiler settings.

ALTERNATIVE IN GIT BASH

```
mvn clean compile -e -X
```

5. Java Version Compatibility Resolution

IN ECLIPSE IDE → POM.XML TAB

Open `pom.xml`, click the `pom.xml` source tab at the bottom, and ensure Java 17 properties are configured:

```
<properties>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>
```

Press **Ctrl + S** to save. Then right-click project → **Properties** → **Java Build Path** → **Libraries** → verify **JRE System Library** matches JavaSE-17.

6. Skipping Tests During Packaging & Associated Risks

IN ECLIPSE IDE

Right-click project → **Run As** → **Maven build...** → in Goals type `clean package` → tick **Skip Tests** → click **Run**.

IN GIT BASH

```
mvn clean package -DskipTests
```

Risks: Bypassing tests allows broken business calculations, syntax errors, or unverified database mappings to be deployed directly into production.

7. Identifying and Resolving Dependency Conflicts

IN ECLIPSE IDE

1. Double-click `pom.xml` → click the **Dependency Hierarchy** tab at the bottom.
2. Type the conflicting library in the *Filter* box to locate versions flagged with *omitted for conflict with...*
3. Switch to the `pom.xml` source tab and add an `<exclusion>` tag under the dependency pulling in the obsolete version:

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-web</artifactId>
  <version>5.3.30</version>
  <exclusions>
    <exclusion>
      <groupId>commons-logging</groupId>
      <artifactId>commons-logging</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

8. Unexpected Behavior After Dependency Modification

IN GIT BASH

```
mvn dependency:purge-local-repository
mvn clean install
```

9. "BUILD SUCCESS" but Application Fails to Open in Browser

1. In **Git Bash**: Verify port 8080 is listening: `netstat -ano | findstr :8080`.
2. In **Browser**: Navigate to `http://localhost:8080/HMS/` (context path matching the WAR filename) instead of root `/`.
3. In **Eclipse Console**: Check Tomcat logs for `ServletException` or context initialization errors.

10. Build Plugin Cannot Be Found or Executed

IN ECLIPSE IDE → POM.XML TAB

Verify that the plugin is declared inside `<build><plugins>` (and not only in `<pluginManagement>`), has a valid `<version>`, and its `<executions>` are bound to a valid phase.

Part II: Working with Version Control (Git & GitHub) (40 Marks)

EXECUTE ALL COMMANDS IN GIT BASH

1. Initialize & First Commit:

```
git init
git add .
git commit -m "Initial commit for Hospital Management System"
```

2. Branching & Editing:

```
git checkout -b feature/patient-registration-update
# In Eclipse Package Explorer: open src/main/webapp/register.jsp, add mobile field & password validation, save (Ctrl+S)
git status
git add src/main/webapp/register.jsp
git commit -m "Add mobile number field and client-side password validation"
git push -u origin feature/patient-registration-update
```

3. Inspect Unwanted Changes:

```
git diff src/main/webapp/register.jsp
```

Output: Red lines starting with `-` are deleted; green lines with `+` are added.

4. Stage JSP and CSS:

```
git add src/main/webapp/register.jsp src/main/webapp/css/style.css
git status
```

5. Undo Commit Preserving History:

```
git revert HEAD --no-edit
```

6. Unstage File Without Deleting from Disk:

```
git restore --staged src/main/resources/temp-config.env
```

7. Stash Changes for Urgent Branch Switch:

```
git stash push -m "WIP register.jsp modifications"
git checkout hotfix/appointment-booking
# (Fix bug in Eclipse, stage, commit)
git checkout feature/patient-registration-update
git stash pop
```

8. Rebase Feature Branch onto Main:

```
git checkout main
git pull origin main
git checkout feature/patient-registration-update
git rebase main
```

9. Recover Lost Commit:

```
git reflog
git checkout -b recovery-branch <commit_hash>
```

10. Configure Remote & Push:

```
git remote add origin https://github.com/your-username/HMS.git
git branch -M main
git push -u origin main
```

Part III: Run the Project with Docker (20 Marks)

1. Docker Environment Consistency [4M]

Docker encapsulates JDK 17, Apache Tomcat, and application dependencies inside an immutable container, ensuring identical execution across all developers' machines.

2. Create Dockerfile in Eclipse [4M]

IN ECLIPSE IDE

1. In **Package Explorer**, right-click project root folder → **New** → **File**.
2. Enter File name: **Dockerfile** (no file extension) → click **Finish**.
3. Paste the following configuration and save (**Ctrl + S**):

```
FROM tomcat:9.0-jdk17-temurin
WORKDIR /usr/local/tomcat/webapps/
RUN rm -rf /usr/local/tomcat/webapps/*
COPY target/HMS.war ./ROOT.war
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

3, 4 & 5. Build, Run, and Push Container [12M]

STEP 1: OPEN DOCKER DESKTOP (ENSURE ENGINE IS RUNNING)

STEP 2: IN GIT BASH

```
# Package the WAR first in Eclipse or Git Bash
mvn clean package -DskipTests

# Build Docker image
docker build -t hms:v1 .

# Run container mapping host port 8080 to container port 8080
docker run -d -p 8080:8080 --name hms-container hms:v1

# Verify container is active
docker ps
```

STEP 3: IN WEB BROWSER

Visit <http://localhost:8080> to test the running application.

STEP 4: PUSH TO DOCKER HUB

```
docker login
docker tag hms:v1 your_dockerhub_username/hms:v1
docker push your_dockerhub_username/hms:v1
```

SET 2: E-Ticketing System (OTS)

Subject Code: 23CC505PC | **Faculty:** K. Yasaswini | **Branch:** CSE - A | **Repository:** <https://github.com/ssvkotamraju/E-ticketing.git>

Part I: Maven Build Execution & Troubleshooting (40 Marks)

- IN GIT BASH:** `git clone https://github.com/ssvkotamraju/E-ticketing.git`.
- IN ECLIPSE:** `File` → `Import...` → `Maven` → `Existing Maven Projects` → browse to folder → click `Finish`.
- IN ECLIPSE → POM.XML TAB:** In `pom.xml`, update `<maven.compiler.source>17</maven.compiler.source>` and `<maven.compiler.target>17</maven.compiler.target>`. Save (Ctrl+S).
- IN GIT BASH:** `mvn clean compile -e`. Scan output for `[ERROR] COMPILATION ERROR` to fix missing imports or syntax bugs.
- IN ECLIPSE → POM.XML TAB:** Locate `<packaging>` and change from `jar` to `<packaging>war</packaging>`.
- Clean Build Utility:** Running `mvn clean package` deletes old compiled class files in `target/`, preventing stale code issues.
- Browser Troubleshooting:** Check if container is running (`docker ps`); verify port 8080 is open; access `http://localhost:8080/E-ticketing/`.
- IN GIT BASH:** Skip tests during package: `mvn clean package -DskipTests`.
- Download Errors Diagnosis:** Verify Central coordinates; check `~/.m2/repository` for corrupted `.lastUpdated` files; inspect proxy in `settings.xml`.
- IN GIT BASH:** Run test suite: `mvn test`.

Part II: Working with Version Control (Git & GitHub) (40 Marks)

IN GIT BASH

```
# Initial Commit on local folder
git init && git add . && git commit -m "Initial commit for Railway Ticket Booking System"

# Create Feature Branch
git checkout -b feature/train-search

# Status & Diff
git status && git diff

# Discard Uncommitted Changes in train-search.jsp
git restore src/main/webapp/train-search.jsp

# Reset Local Commits
git reset --hard <commit_hash>

# Delete Merged Branch
git branch --merged && git branch -d feature/old-branch && git push origin --delete feature/old-branch

# Merge Feature Branches into Main
git checkout main && git merge feature/train-search && git merge feature/passenger-registration

# Create & Apply Git Patch
git format-patch -1 HEAD --stdout > feature.patch
git apply --stat feature.patch
git apply feature.patch

# Remote Configuration & Push
git remote add origin https://github.com/username/railway-ticketing.git
git branch -M main && git push -u origin main
```

Part III: Run the Project with Docker (20 Marks)

IN ECLIPSE: CREATE DOCKERFILE IN PROJECT ROOT

```
FROM tomcat:9.0-jdk17-temurin
WORKDIR /usr/local/tomcat/webapps/
RUN rm -rf /usr/local/tomcat/webapps/*
COPY target/E-ticketing.war ./ROOT.war
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

OPEN DOCKER DESKTOP

IN GIT BASH:

```
mvn clean package -DskipTests
docker build -t ots:v1 .
docker run -d -p 8080:8080 --name ots-container ots:v1
docker ps
```

SET 3: Online Recruitment System (ORS)

Subject Code: 23CC505PC | Faculty: K. Yasaswini | Branch: CSE - A | Repository: <https://github.com/ssvkotamraju/ORS.git>

Part I: Maven Build & Execution (40 Marks)

- IN GIT BASH:** `git clone https://github.com/ssvkotamraju/ORS.git`.
IN ECLIPSE: File → Import... → Maven → Existing Maven Projects.
- IN ECLIPSE:** Check Problems View for missing dependencies; fix in `pom.xml`; right-click project → Maven → Update Project.
- IN GIT BASH:** `mvn dependency:tree -Dverbose`. Add `<exclusions>` in `pom.xml` to remove conflicting transitive dependencies.
- IN ECLIPSE → POM.XML TAB:** Set compiler properties to `17`; in Project Properties → Java Build Path, ensure JDK 17 is selected.
- IN GIT BASH:** `mvn clean install -U`
- Port Settings:** In `pom.xml` inside `<tomcat7-maven-plugin <configuration>`, set `<port>8082</port>`. Access `http://localhost:8082/ORS`.
- Kill Port 8080 in Git Bash / PowerShell:**

```
netstat -ano | findstr :8080
taskkill //PID <PID> //F
```
- IN ECLIPSE → POM.XML TAB:** Configure `maven-compiler-plugin` with `<release>17</release>`.
- Dependency Failure Diagnosis:** Check Central coordinates, check proxy in `~/.m2/settings.xml`, delete corrupted `.m2/repository` folders.
- IN GIT BASH:** `mvn -v && mvn dependency:tree`

Part II: Git Rebase & SSH Key Workflow (40 Marks)

IN GIT BASH:

```
# Clone via SSH & Branch
git clone git@github.com:ssvkotamraju/ORS.git
cd ORS && git checkout -b feature/apply-job

# Fetch & Rebase
git fetch origin
git rebase origin/main

# Resolve Conflict: Open src/main/webapp/apply.jsp in Eclipse, edit conflict markers, save (Ctrl+S)
git add src/main/webapp/apply.jsp
git rebase --continue

# Restore Missing Commit & Abort on Error
git reflog && git cherry-pick <commit_hash>
git rebase --abort

# SSH Key Setup
ssh-keygen -t ed25519 -C "student@kmit.in"
cat ~/.ssh/id_ed25519.pub
# Copy key & add in GitHub -> Settings -> SSH and GPG Keys
ssh -T git@github.com

# Push After Rebase
git push --force-with-lease origin feature/apply-job
```

Part III: Run the Project with Docker (20 Marks)

OPEN DOCKER DESKTOP

IN GIT BASH:

```
mvn clean package -DskipTests
docker build -t ors:latest .
docker run -d -p 8080:8080 --name ors-container ors:latest
docker tag ors:latest your_dockerhub_username/ors:latest
docker login && docker push your_dockerhub_username/ors:latest
```

SET 4: Online Shopping Application

Subject Code: 23CC501PC | **Faculty:** M. Nikitha | **Branch:** CSE - B | **Repository:** <https://github.com/nikithamoturi/OnlineShoppingApp.git>

Part I: Maven Application Development (40 Marks)

Q1. Setup and Build

- IN GIT BASH:** `git clone https://github.com/nikithamoturi/OnlineShoppingApp.git`.
IN ECLIPSE: File → Import... → Maven → Existing Maven Projects.
- `pom.xml` coordinates and JSP views define app build and presentation. Dependencies: `javax.servlet-api` and `maven-compiler-plugin`.
- IN ECLIPSE → POM.XML TAB:** If `failOnMissingWebXml` error triggers, add `<failOnMissingWebXml>false</failOnMissingWebXml>` under `<maven-war-plugin configuration>`. Save (Ctrl + S).
- IN BROWSER:** Test `http://localhost:8080/OnlineShoppingApp/products.jsp`.

Q2. Dependency and Build Troubleshooting

- IN GIT BASH:** `mvn dependency:tree`
- Resolution: Maven follows the "Nearest-Wins" rule in the dependency tree hierarchy.
- Local cache is stored in `~/.m2/repository`.
- Debug commands: `mvn clean compile`, `mvn clean package -X -e`.
- Isolate issues: If Maven build succeeds, check Tomcat `catalina.out` logs for runtime errors (404/500).

Part II: Git & GitHub Integration (40 Marks)

IN GIT BASH:

```
# Q3. Clone, Branch, Edit, Stage & Commit
git clone https://github.com/nikithamoturi/OnlineShoppingApp.git
cd OnlineShoppingApp
git checkout -b feature/product-update
# (In Eclipse Package Explorer: open src/main/webapp/products.jsp, add Category & Quantity, Ctrl+S)
git diff src/main/webapp/products.jsp
git add src/main/webapp/products.jsp
git commit -m "Add Product Category and Quantity in products.jsp"

# Q4. Sync and Merge
git fetch origin
git diff main...feature/product-update
git checkout main
git pull origin main
git merge feature/product-update
git push origin main
```

Part III: Containerization and Deployment (20 Marks)

OPEN DOCKER DESKTOP

IN GIT BASH:

```
docker build -t onlineshopping:v1 .
docker run -d -p 8080:8080 --name onlineshopping-app onlineshopping:v1
docker tag onlineshopping:v1 your_dockerhub_username/onlineshopping:v1
docker login && docker push your_dockerhub_username/onlineshopping:v1
```

SET 5: Blood Bank Management System

Subject Code: 23CC505PC | **Faculty:** S. Srija | **Branch:** CSE | **Repository:** <https://github.com/sarasrija/Blood-Bank-Management-system.git>

Part I: Git & GitHub Collaboration (20 Tasks = 40 Marks)

EXECUTE ALL 20 COMMANDS IN GIT BASH

#	Task / Scenario	Exact Command to Type in Git Bash
1	Clone codebase locally	<code>git clone https://github.com/sarasrija/Blood-Bank-Management-system.git</code>
2	Verify remote URLs	<code>git remote -v</code>
3	Inspect staged/unstaged status	<code>git status</code>
4	Create and switch to branch	<code>git checkout -b feature/donor-registration</code>
5	List all branches & confirm current	<code>git branch -a</code>
6	Commit <code>DonorServlet.java</code>	<code>git add src/main/java/com/bloodbank/servlet/DonorServlet.java && git commit -m "Add donor registration servlet"</code>
7	Amend commit with <code>web.xml</code>	<code>git add src/main/webapp/WEB-INF/web.xml && git commit --amend --no-edit</code>
8	Fetch remote updates without merging	<code>git fetch origin</code>
9	Merge remote main into active branch	<code>git merge origin/main</code>
10	Rebase feature branch onto main	<code>git rebase main</code>
11	Cancel rebase on conflict	<code>git rebase --abort</code>
12	Revert commit in <code>DonorTest.java</code>	<code>git revert <commit_hash> --no-edit</code>
13	Undo commit keeping edits staged	<code>git reset --soft HEAD~1</code>
14	Untrack file keeping on disk	<code>git rm --cached src/main/resources/db-config.env</code>
15	Stash unfinished work	<code>git stash push -m "WIP inventory.jsp modifications"</code>
16	List and restore stashed work	<code>git stash list && git stash pop</code>
17	Diff <code>index.jsp</code> against main	<code>git diff main..feature/donor-registration -- src/main/webapp/index.jsp</code>
18	Compact visual branch graph	<code>git log --oneline --graph --decorate --all</code>
19	Return to main & merge feature	<code>git checkout main && git merge feature/donor-registration</code>
20	Push main and verify status	<code>git push origin main && git status</code>

Part II: Maven Java Application Development (40 Marks)

- Q1: **IN ECLIPSE → POM.XML TAB**: Fix XML schema tags; add `tomcat7-maven-plugin` under `<build><plugins>`.
- Q2: **IN POM.XML**: Add MySQL dependency `mysql-connector-j`; add JUnit with `<scope>test</scope>`.
- Q3: Target WAR: `target/Blood-Bank-Management-system-0.0.1-SNAPSHOT.war`; Context URL: `http://localhost:8080/bloodbank`.
- Q4: `<pluginManagement>` acts as a shared configuration template; `<plugins>` triggers execution. SNAPSHOT enables mutable development builds.
- Q5: **IN GIT BASH:** `mvn -v`; `mvn clean package -X`.
- Q6: **IN GIT BASH:**

```
mvn install:install-file -Dfile=blood-analytics.jar -DgroupId=com.vendor.analytics -DartifactId=blood-analytics -Dversion=1.0.0 -Dpackaging=jar
mvn dependency:tree
```

- Q7: Run single test: `mvn test -Dtest=BloodInventoryTest`. Ignore errors: `-Dmaven.test.failure.ignore=true`.
- Q8: Unused dependency analysis: `mvn dependency:analyze`. `provided` scope ensures servlet API is provided by Tomcat container at runtime.

Part III: Dockerization and Container Management (20 Marks)

OPEN DOCKER DESKTOP

IN GIT BASH:

1. `git clone https://github.com/sarasrija/Blood-Bank-Management-system.git && cd Blood-Bank-Management-system && ls -la`
2. **IN ECLIPSE:** Create `Dockerfile` in project root:

```
FROM openjdk:17-jdk-slim
WORKDIR /app
COPY target/*.war /app/Blood-managent.war
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/Blood-managent.war"]
```

3. `docker build -t Bloodbankapp-image .`
4. `docker run -d -p 8080:8080 --name Bloodbank-app-container Bloodbankapp-image`
5. `docker ps`
6. `docker ps -a`

7. `docker exec -it Bloodbank-app-container /bin/bash`
8. `docker stop Bloodbank-app-container && docker start Bloodbank-app-container`
9. `docker commit 0e993d2009a1 your_dockerhub_username/Bloodbankapp:v1`
10. `docker login`
11. `docker push your_dockerhub_username/Bloodbankapp:v1`
12. `docker logout`

SET 6: Sports Management System

Subject Code: 23CC505PC | **Faculty:** S. Srija | **Branch:** CSE | **Repository:** <https://github.com/sarasrija/SportManagementSystem.git>

Part I: Git & GitHub Collaboration (20 Tasks = 40 Marks)

EXECUTE ALL 20 COMMANDS IN GIT BASH

1. `git clone https://github.com/sarasrija/SportManagementSystem.git`
2. `git remote -v`
3. `git status`
4. `git checkout -b feature/player-registration`
5. `git branch -a`
6. `git add src/main/java/com/sports/servlet/RegistrationServlet.java && git commit -m "Add player registration servlet"`
7. `git add src/main/webapp/WEB-INF/web.xml && git commit --amend --no-edit`
8. `git fetch origin`
9. `git merge origin/main`
10. `git rebase main`
11. `git rebase --abort`
12. `git revert <commit_hash> --no-edit`
13. `git reset --soft HEAD~1`
14. `git rm --cached src/main/resources/db-config.env`
15. `git stash push -m "WIP schedule.jsp modifications"`
16. `git stash list && git stash pop`
17. `git diff main..feature/player-registration -- src/main/webapp/index.jsp`
18. `git log --oneline --graph --decorate --all`
19. `git checkout main && git merge feature/player-registration`
20. `git push origin main && git status`

Part II: Maven Java Application Development (40 Marks)

- Q1: **IN ECLIPSE → POM.XML TAB:** Fix schema tags; add `tomcat7-maven-plugin` under `<build><plugins>`:

```
<plugin>
  <groupId>org.apache.tomcat.maven</groupId>
  <artifactId>tomcat7-maven-plugin</artifactId>
  <version>2.2</version>
  <configuration>
    <port>8080</port>
    <path>/sports</path>
  </configuration>
</plugin>
```

- Q2: **IN POM.XML:** Add MySQL dependency; add JUnit dependency with `<scope>test</scope>`.
- Q3: Target artifact: `target/sports-management.war`; Context URL: `http://localhost:8080/sports`.
- Q4: `<pluginManagement>` defines templates; `<plugins>` executes goals. SNAPSHOT represents active development.
- Q5: **IN GIT BASH:** `mvn -v`; `mvn clean package -X`.
- Q6: **IN GIT BASH:**

```
mvn install:install-file -Dfile=sports-analytics.jar -DgroupId=com.vendor -DartifactId=sports-analytics -
Dversion=1.0.0 -Dpackaging=jar
mvn dependency:tree
```

- Q7: Run single test: `mvn test -Dtest=TournamentBracketTest`. Ignore errors: `-Dmaven.test.failure.ignore=true`.
- Q8: Unused dependency analysis: `mvn dependency:analyze`. Provided scope ensures Tomcat provides Servlet runtime classes.

Part III: Dockerization and Container Management (20 Marks)

OPEN DOCKER DESKTOP

IN GIT BASH:

1. `git clone https://github.com/sarasrija/SportManagementSystem.git && cd SportManagementSystem && ls -la`
2. **IN ECLIPSE:** Create `Dockerfile` in project root:

```
FROM openjdk:17-jdk-slim
WORKDIR /app
COPY target/sports-management.jar /app/sports-management.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "sports-management.jar"]
```
3. `docker build -t sportsapp-image .`
4. `docker run -d -p 8080:8080 --name sports-app-container sportsapp-image`
5. `docker ps`
6. `docker ps -a`
7. `docker exec -it sports-app-container /bin/bash`
8. `docker stop sports-app-container && docker start sports-app-container`
9. `docker commit 0e993d2009a1 your_dockerhub_username/sportsapp:v1`
10. `docker login`
11. `docker push your_dockerhub_username/sportsapp:v1`
12. `docker logout`

SET 7: Apartment Management System

Subject Code: 23CC505PC | Faculty: S. Srija | Branch: CSE | Repository: <https://github.com/sarasrija/ApartmentManagementSystem.git>

Part I: Git & GitHub Collaboration (20 Tasks = 40 Marks)

EXECUTE ALL 20 COMMANDS IN GIT BASH

1. `git clone https://github.com/sarasrija/ApartmentManagementSystem.git`
2. `git remote -v`
3. `git status`
4. `git checkout -b feature/tenant-registration`
5. `git branch -a`
6. `git add src/main/java/com/apartment/servlet/TenantServlet.java && git commit -m "Add tenant registration servlet"`
7. `git add src/main/webapp/WEB-INF/web.xml && git commit --amend --no-edit`
8. `git fetch origin`
9. `git merge origin/main`
10. `git rebase main`
11. `git rebase --abort`
12. `git revert <commit_hash> --no-edit`
13. `git reset --soft HEAD~1`
14. `git rm --cached src/main/resources/db-config.env`
15. `git stash push -m "WIP complaints.jsp modifications"`
16. `git stash list && git stash pop`
17. `git diff main..feature/tenant-registration -- src/main/webapp/index.jsp`
18. `git log --oneline --graph --decorate --all`
19. `git checkout main && git merge feature/tenant-registration`
20. `git push origin main && git status`

Part II: Maven Java Application Development (40 Marks)

- Q1: **IN ECLIPSE → POM.XML TAB:** Fix XML element structure; add `tomcat7-maven-plugin` under `<build><plugins>`.
- Q2: **IN POM.XML:** Add MySQL dependency; add JUnit test dependency with `<scope>test</scope>`.
- Q3: Target artifact: `target/apartment-management.war`; Context URL: `http://localhost:8080/apartment`.
- Q4: `<pluginManagement>` config template vs `<plugins>` execution; SNAPSHOT dynamic check vs Release immutable.
- Q5: **IN GIT BASH:** `mvn -v`; `mvn clean package -X`.
- Q6: **IN GIT BASH:**

```
mvn install:install-file -Dfile=apartment-analytics.jar -DgroupId=com.vendor -DartifactId=apartment-analytics -Dversion=1.0.0 -Dpackaging=jar
mvn dependency:tree
```

- Q7: Run single test: `mvn test -Dtest=MaintenanceBillingTest`. Skip on failure: `-Dmaven.test.failure.ignore=true`.
- Q8: Detect unused: `mvn dependency:analyze`. Provided scope ensures servlet-api is inherited from Tomcat container.

Part III: Dockerization and Container Management (20 Marks)

OPEN DOCKER DESKTOP

IN GIT BASH:

1. `git clone https://github.com/sarasrija/ApartmentManagementSystem.git && cd ApartmentManagementSystem && ls -la`
2. **IN ECLIPSE:** Create `Dockerfile` in project root:

```
FROM openjdk:17-jdk-slim
WORKDIR /app
COPY target/apartment-management.jar /app/apartment-management.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "apartment-management.jar"]
```

3. `docker build -t apartmentapp-image .`
4. `docker run -d -p 8080:8080 --name apartment-app-container apartmentapp-image`
5. `docker ps`
6. `docker ps -a`
7. `docker exec -it apartment-app-container /bin/bash`
8. `docker stop apartment-app-container && docker start apartment-app-container`
9. `docker commit 0e993d2009a1 your_dockerhub_username/apartmentapp:v1`
10. `docker login`
11. `docker push your_dockerhub_username/apartmentapp:v1`
12. `docker logout`

SET 8: Vehicle Rental Management System

Subject Code: 23CC501PC | Faculty: M. Nikitha | Branch: CSE - B | Repository: <https://github.com/nikithamoturi/VehicleRentalManagementSystem.git>

Part I: Maven Web Project Development (40 Marks)

Q1. Project Setup and Build

1. **IN GIT BASH:** `git clone https://github.com/nikithamoturi/VehicleRentalManagementSystem.git`.
IN ECLIPSE: `File → Import... → Maven → Existing Maven Projects`. Verify JSPs under `src/main/webapp/`: `index.jsp`, `login.jsp`, `register.jsp`, `vehicle.jsp`.
2. Coordinates: `groupId`, `artifactId`, `version`, `packaging` (war). Dependencies: `javax.servlet-api` and `jstl`.
3. **IN ECLIPSE → POM.XML TAB:** If `failOnMissingWebXml` error occurs, configure `maven-war-plugin` with `<failOnMissingWebXml>false</failOnMissingWebXml>`. Save (Ctrl+S).
4. **IN BROWSER:** Access `http://localhost:8080/VehicleRentalManagementSystem/vehicle.jsp`.

Q2. Dependency & Build Troubleshooting

1. **IN GIT BASH:** `mvn dependency:tree`
2. Dependency conflicts resolved using "Nearest-Definition" strategy.
3. Dependencies stored in `~/.m2/repository`.
4. Troubleshooting commands: `mvn clean`, `mvn compile`, `mvn package -X`.
5. If build passes but app fails: check Tomcat port bind issues, database connectivity, and JSP scriptlet compilation in `catalina.out`.

Part II: Git & GitHub Integration (40 Marks)

IN GIT BASH:

```
# Q3. Clone, Branch, Edit, Stage & Commit
git clone https://github.com/nikithamoturi/VehicleRentalManagementSystem.git
cd VehicleRentalManagementSystem
git checkout -b feature/vehicle-update
# (In Eclipse Package Explorer: open src/main/webapp/vehicle.jsp, add Vehicle Type & Availability Status, Ctrl+S)
git diff src/main/webapp/vehicle.jsp
git add src/main/webapp/vehicle.jsp
git commit -m "Add Vehicle Type and Availability Status in vehicle.jsp"

# Q4. Sync and Merge
git fetch origin
git diff main...feature/vehicle-update
git checkout main
git pull origin main
git merge feature/vehicle-update
git push origin main
```

Part III: Dockerization and Container Management (20 Marks)

OPEN DOCKER DESKTOP

IN GIT BASH:

```
# Build and Run
docker build -t vehiclerentalapp:v1 .
docker run -d -p 8080:8080 --name vehiclerental-container vehiclerentalapp:v1

# Tag & Push to Docker Hub
docker tag vehiclerentalapp:v1 your_dockerhub_username/vehiclerentalapp:v1
docker login
docker push your_dockerhub_username/vehiclerentalapp:v1
```

SET 9: Gym Membership Management System

Subject Code: 23CC501PC | Faculty: M. Nikitha | Branch: CSE - B | Repository: <https://github.com/nikithamoturi/GymManagementSystem.git>

Part I: Maven Web Project Development (40 Marks)

Q1. Configuration and Build

1. **IN GIT BASH:** `git clone https://github.com/nikithamoturi/GymManagementSystem.git`.
IN ECLIPSE: `File → Import... → Maven → Existing Maven Projects`.
2. Explain elements: `groupId` (domain package), `artifactId` (project ID), `version` (iteration), `packaging` (WAR), `dependencies` (external libraries).
3. **IN GIT BASH:** `mvn clean package`. Traverses `validate → compile → test → package`. Output: `target/GymManagementSystem.war`.
4. **IN ECLIPSE → POM.XML TAB:** Change `<finalName>GymMembershipApp</finalName>` under `<build>`. Re-run `mvn clean package`.

Q2. Dependency & Build Analysis

1. **IN GIT BASH:** `mvn dependency:tree`. Dependencies: `javax.servlet-api` and `jstl`.
2. Difference: Dependency is a runtime library in classpath; Plugin is a build tool executing lifecycle goals.
3. Commands: Clean (`mvn clean`), Compile (`mvn compile`), Package (`mvn package`), Skip tests (`mvn package -DskipTests`).
4. Failure on another system: Check offline mode, proxy blocking Central, or corrupt `~/.m2` files. Fix via `mvn clean compile -U`.

Part II: Git & GitHub Integration (40 Marks)

IN GIT BASH:

```
# Q3. Clone, Branch, Edit, Stage & Commit
git clone https://github.com/nikithamoturi/GymManagementSystem.git
cd GymManagementSystem
git checkout -b feature/membership-plans
# (In Eclipse Package Explorer: open src/main/webapp/plans.jsp, add Membership Type & Status, Ctrl+S)
git diff src/main/webapp/plans.jsp
git add src/main/webapp/plans.jsp
git commit -m "Add Membership Type and Membership Status"

# Q4. Sync, Undo & Merge
git fetch origin
git rebase origin/main
# Undo commit without deleting files:
git reset --soft HEAD~1
# Merge into main and push:
git checkout main
git pull origin main
git merge feature/membership-plans
git push origin main
```

Part III: Dockerization and Container Management (20 Marks)

OPEN DOCKER DESKTOP

IN GIT BASH:

```
# Package WAR and Build Image
mvn clean package -DskipTests
docker build -t gymapp:v1 .

# Run Container
docker run -d -p 8080:8080 --name gym-container gymapp:v1
docker ps
docker logs -f gym-container

# Tag & Push to Docker Hub
docker tag gymapp:v1 your_dockerhub_username/gymmanagementsystem:v1
docker login
docker push your_dockerhub_username/gymmanagementsystem:v1
```